

Universidad San Carlos de Guatemala
Facultad de ingeniería
Ingeniería en ciencias y sistemas
Manejo e Implementación de Archivos

Manual Técnico C++ DISK ONLINE

Selvin Haroldo Ocoix Maldonado

201944362

Contenido

Resumen	3
Requisitos del Sistema.....	4
Requisitos de hardware	4
Requisitos de Software	5
Descripción del sistema	5
Arquitectura del sistema	6
Frontend (React):	6
Backend (C++):.....	6
Estructuras del Sistema (EXT3).....	9
Flujo de ejecución.....	10
Conclusión.....	11

Resumen

El proyecto C++ Disk Online consiste en el desarrollo de un manejador de sistemas de archivos simulados con tipado nativo en C++, soportando la arquitectura transaccional de **EXT3**. Este sistema incluye la simulación de particiones en un archivo binario .dk, algoritmos de asignación de bloques y protección de consistencia mediante Journaling, integrándose a una sofisticada arquitectura web para facilitar su administración gráfica.

Requisitos del Sistema

Requisitos de hardware

El sistema no requiere hardware físico especializado ya que la lógica opera nativamente, pero en entornos de producción exige computo distribuido.

Hardware mínimo:

Recurso	Requisito mínimo
Procesador	Ryzen 3 o equivalente
Memoria RAM	4 GB
Almacenamiento	500 MB libres

Hardware recomendado:

Recurso	Recomendado
Procesador	Ryzen 5 o superior
RAM	8 GB
Almacenamiento	SSD

Requisitos de Software

Software	Versión recomendada	Uso
G++ (GCC)	C++17	Lenguaje principal
Visual Code	28 o inferior	Entorno de desarrollo
React	v18.0+	Interfaz Gráfica
AWS EC2	S3	Ejecución Cloud
Sistema Operativo	Linux	Plataforma de ejecución

Descripción del sistema

C++ DISK ONLINE permite:

- Implementar un motor de discos simulados administrando un MBR (Master Boot Record) y particiones nativas binarias.
- Diseñar y mantener la coherencia de datos a través de una superestructura EXT3, utilizando inodos y bloques matemáticamente controlados.
- Validar reglas de autenticación y lectura de superbloques mediante la creación de un modal Login gráfico.
- Ofrecer una vista jerárquica instantánea de carpetas operando como un explorador interactivo estático.

Arquitectura del sistema

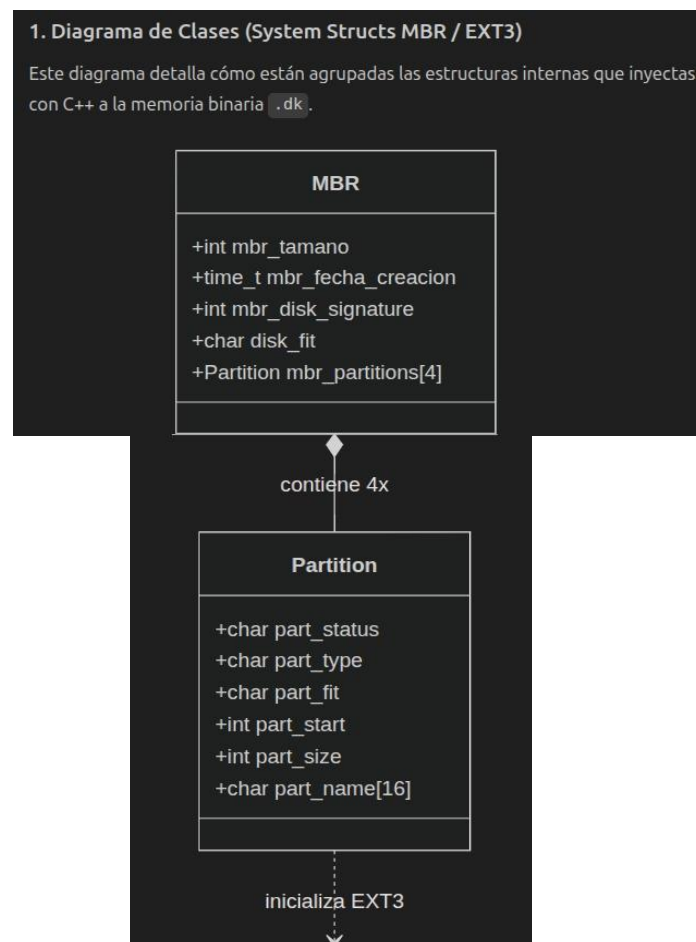
El emulador opera estructurado en un modelo Cliente-Servidor (Cloud-Native) basado en Amazon Web Services:

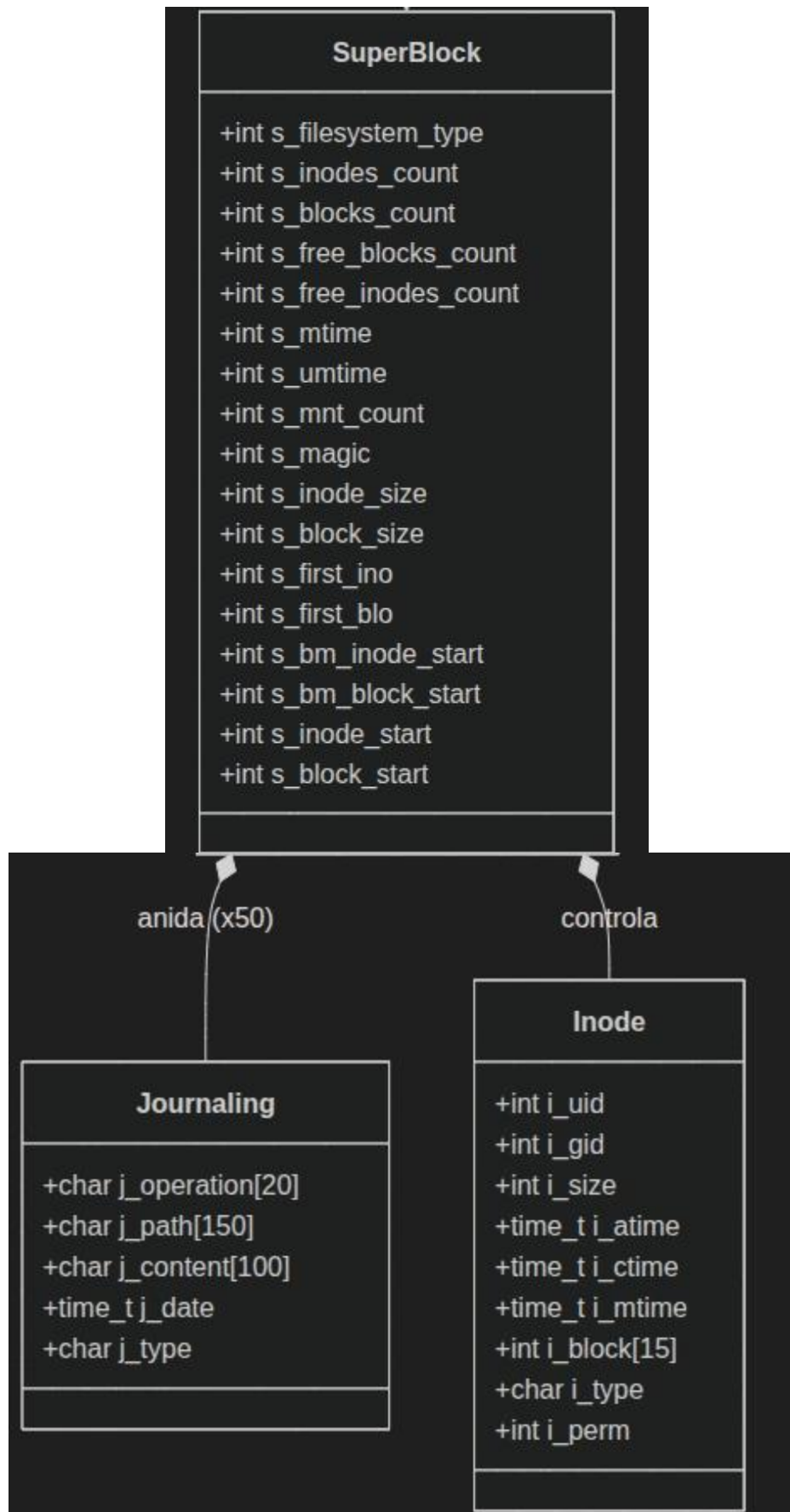
Frontend (React):

Interfaz web desarrollada con ReactJS y estilizada con CSS. Provee un diseño moderno mediante Glassmorphism. La comunicación se realiza enviando comandos texto a través de peticiones HTTP POST estructuradas mediante Axios.

Backend (C++):

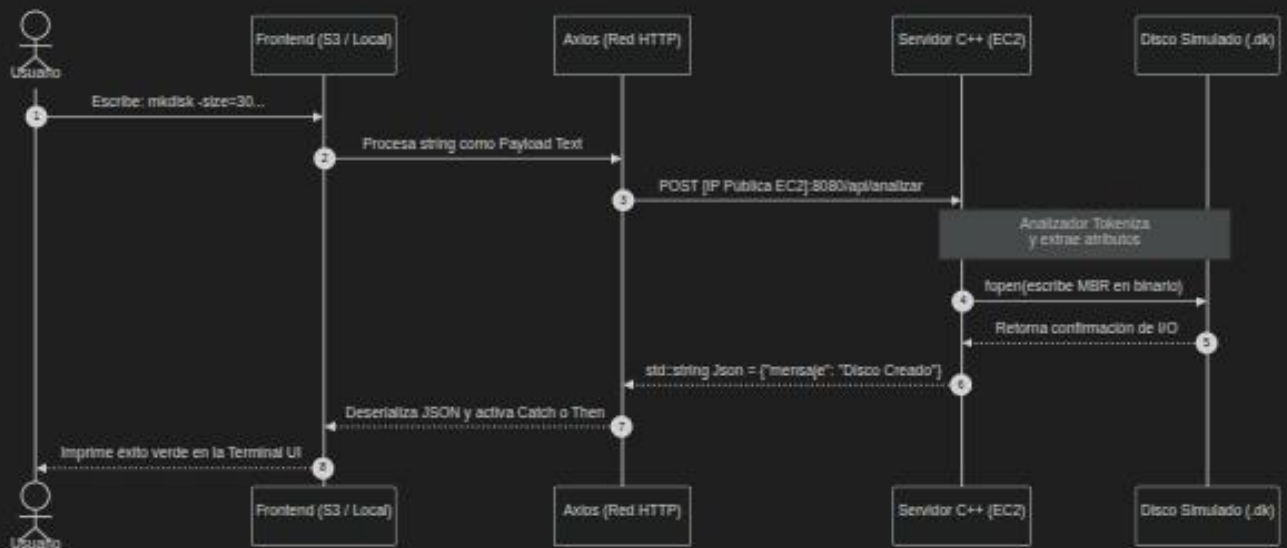
Desarrollado en C++17 empleando cpp-httplib. Recibe los comandos fdisk, mkfs, entre otros; formatea o inyecta data al disco local, y responde emitiendo una confirmación JSON al cliente.





2. Diagrama de Secuencia (Arquitectura Cliente-Servidor AWS)

Demuestra paso a paso cómo recorre un comando desde tu PC, pasa por React en S3, y aterriza en C++ en EC2.



Estructuras del Sistema (EXT3)

El procesamiento y empaquetado binario se divide en fases estrictas sobre punteros `std::fstream`.

Journaling (Journaling Struct):

Estructura crítica en `estructuras.h` encargada de asegurar atomicidad de la información. Evita colapsos eléctricos registrando la ruta y fecha de operación. Ocupa un factor estático de 50 repeticiones tras el Superbloque.

```
cpp

/* Archivo: backend/src/utils.cpp */
std::vector<std::string> tokenizarLinea(const std::string& linea) {
    std::vector<std::string> tokens;
    std::string actual = "";
    bool dentroDeComillas = false;
    for (char c : linea) {
        if (c == '"') {
            dentroDeComillas = !dentroDeComillas;
            actual += c;
        } else if (c == ' ' && !dentroDeComillas) {
            if (!actual.empty()) { tokens.push_back(actual); actual = ""; }
        } else { actual += c; }
    }
    if (!actual.empty()) tokens.push_back(actual);
    return tokens;
}
```

Gestión y Despeje (n):

Para formular particiones limpias, se calcula el despeje de inodos con la siguiente lógica en `comandos_ext2.cpp`:

```
double espacioSobrante = partTarget.part_size - sizeof(SuperBlock) - (50 *
sizeof(Journaling));
```

```
cpp

/* Archivo: backend/src/analizador.cpp */
if (comandoPrincipal.find("#") != std::string::npos || comandoPrincipal == "PAUSE") {
    return "{\nmensaje\": \"Comentario detectado y omitido exitosamente.\n}";
}
if (comandoPrincipal == "MKDISK") return ejecutarMkdisk(parametrosEncontrados);
if (comandoPrincipal == "LOGIN") return ejecutarLogin(parametrosEncontrados);
```

Flujo de ejecución

El ciclo de vida de un comando introducido en la interfaz es el siguiente:

1. Ingreso y Captura: El Frontend lee el texto de la terminal y ejecuta una petición HTTP POST al puerto 8080 de EC2.
2. Tokenización (Analizador): En el backend, C++ rompe la cadena detectando palabras reservadas como `-size=` separando al sistema en tuplas ordenadas.
3. Escritura Binaria: Dependiendo del comando apuntado, C++ realiza `reinterpret_cast` inyectando en la posición exacta del puntero en el disco simulado.
4. Respuesta: El backend empaceta la descripción de éxito validada en formato JSON puro y la devuelve en la red.
5. Render de Interfaz: React actualiza sus Estados (`useState`) provocando cambios automáticos a la vista sin recargar la página.

Conclusión

El desarrollo de este Emulador de Archivos ha demostrado con éxito la aplicación práctica de operaciones críticas de bajo nivel C++, abarcando la manipulación segura de punteros binarios (fread/fwrite), combinados agresivamente a través de un puente HTTP moderno y consumido por un Frontend altamente estético desplegado exitosamente en infraestructuras de Nube (AWS)..